



ISLA Santarém – Instituto Politécnico

Curso Tesp em Tecnologias e Programação de Sistemas de Informação

Sistema de Gestão de Notas

Alexandre Nogueira

Vicente Dias

Docente:

Luís Beirante

Unidade Curricular: Programação de Computadores II

Santarém

Ano letivo 2025/2026

INDICE

INTRODUÇÃO	4
ENQUADRAMENTO DO PROBLEMA	4
DESCRIÇÃO DA APLICAÇÃO	4
DESENVOLVIMENTO	6
PERSISTÊNCIA DE DADOS.....	9
PESQUISA ORDENÇÃO E ESTATÍSTICA.....	9
TESTES REALIZADOS	10
DECLARAÇÃO DE USO DE IAGEN	11
CONCLUSÃO	11

INTRODUÇÃO

Neste trabalho desenvolvemos um sistema de gestão de notas escolares com base em python e a biblioteca tkinter para interface. O objetivo principal deste sistema é a atribuição de notas a alunos por disciplinas e a apresentação de relatórios com base nos dados inseridos. Propusemo-nos também a criar uma interface para o sistema, de forma a tornar o seu uso mais intuitivo e para uma melhor e mais rápida interpretação das suas funcionalidades.

ENQUADRAMENTO DO PROBLEMA

Esta aplicação foi pensada para ser usada por um professor, com foco em agilizar a listagem de alunos e a atribuição de notas por aluno, bem como a exposição de relatórios como médias de notas por aluno ou por disciplina. O que esta aplicação pretende resolver é a digitalização da tarefa de atribuição de notas bem como a rápida modificação de listas de alunos e de disciplinas, agrupando num só espaço todas as ferramentas necessárias que um professor possa necessitar para tal efeito, auxiliado por uma interface intuitiva e fácil de usar para qualquer grau de instrução tecnológica.

DESCRIÇÃO DA APLICAÇÃO

Ao iniciar a aplicação, esta dá as boas-vindas ao utilizador e apresenta um menu inicial. Na primeira opção “Gestão de Alunos”, é a janela onde o utilizador pode adicionar; editar; remover; pesquisar ou organizar alfabeticamente os alunos. Esta janela apresenta na parte superior dois campos para preencher “ID” e “Nome” quatro botões e na parte inferior a uma janela de visualização com lista de alunos organizada por ID’s. Para adicionar um aluno basta preencher o campo do nome com o nome do aluno que se pretende adicionar, não permitindo adicionar alunos com o mesmo nome e apresentado uma mensagem de SUCESSO que valida a conclusão da ação. Para editar é necessário escolher o ID que se pretende editar e preencher a edição do nome no campo de nome. Para remover um aluno a seleção de ID também é a condição necessária a preencher. Para pesquisar, podemos pesquisar pelo nome completo do aluno, mas a aplicação aceita também porções de um nome e apresentará um ou mais alunos caso essa porção esteja contida no nome de mais que um aluno. Por fim um botão para ordenar alfabeticamente os nomes dos alunos.

A segunda opção “Gestão de Disciplinas” comporta-se analogamente à Gestão de Alunos, mas neste caso com as disciplinas. Também com a mesma disposição e com a possibilidade de realizar as mesmas ações como para os alunos.

A terceira opção do menu inicial é “Gestão de Notas”. Esta janela já apresenta três campos para preencher, “ID aluno”; “ID disciplina” e “Nota”. Por baixo, encontramos duas janelas lado a lado, uma com a listagem dos alunos e outra com a listagem das disciplinas, para que o utilizador não tenha de decorar os ID’s pretendidos e permitindo ainda que ao utilizador a possibilidade de clicar no nome do aluno ou disciplina e copiar o seu ID diretamente para os campos a preencher. Abaixo, encontramos três botões, “Atribuir Nota”, “Consultar aluno(ID)” e “Consultar disciplina(ID)”, os dois últimos como o botão refere, permite que ao preencher o campo do ID quer do aluno ou disciplina que seja apresentado numa janela de visualização a lista de notas de cada um: por aluno será as notas do aluno a todas as disciplinas, e por disciplinas será as notas de todos os alunos àquela disciplina. Por fim, caso se pretenda atribuir uma nota a dado aluno, o programa requer que os campos de ID aluno e disciplina sejam preenchidos e por fim a nota a atribuir ou a editar.

A quarta opção é “Relatórios” é bastante semelhante à terceira em termos de interface, com exceção de não haver o campo de atribuição de nota e as ações permitidas serem “Média Aluno”; “Média Disciplina” e “Resumo Geral e Ranking”. Portanto utilizando a mesma lógica da opção Gestão de Notas, é possível selecionar um aluno ou disciplina através dos campos para inserir os ID’s, ou clicando no aluno ou disciplina pretendido através da lista apresentada e se se pretender a média quer de um ou outro é mostrada uma pequena mensagem na janela de visualização com a informação da média do dado pretendido. A opção de “Resumo Geral e Ranking” por sua vez, apresenta-nos o total de alunos inseridos, disciplinas e notas atribuídas e por baixo é ainda mostrado um ranking de alunos por média começando pela média mais alta e atribuindo uma posição a cada aluno com notas atribuídas.

A quinta opção “Reiniciar Dados”, como o nome indica limpa todas as listas de dados: alunos, disciplinas e notas.

E por fim a última opção “Sair” é uma forma alternativa de fechar a aplicação.

DESENVOLVIMENTO

A aplicação foi desenvolvida seguindo uma arquitetura modular. O sistema está dividido em componentes independentes, permitindo que a lógica, a persistência de dados e a interface gráfica coexistam sem se sobreporem diretamente. Esta organização facilita a manutenção do código, a deteção de erros e a escalabilidade do sistema.

O projeto é constituído pelos seguintes ficheiros de código fonte:

main.py: Funciona como o ponto de entrada único do sistema. A sua única responsabilidade é invocar a inicialização da interface gráfica.

logica.py: Contém as regras de lógicas da aplicação, a gestão do estado global em memória, as funções de processamento estatístico e as operações CRUD (Create, Read, Update, Delete) dos dados.

interface.py: Implementa toda a interface gráfica de utilizador recorrendo à biblioteca tkinter. É responsável por capturar os eventos do utilizador e exibir as respostas do sistema.

ficheiros.py: Contém as funções responsáveis pela manipulação direta dos ficheiros em formato CSV, garantindo que os dados não se perdem ao fechar a aplicação.

testes.py: Módulo de testes automatizados baseado em pytest. Garante a integridade das funções lógicas e valida o comportamento esperado do sistema face à execução de ações corretas e incorretas.

Para gerir o estado da aplicação em memória de forma eficiente durante a execução, foram utilizadas listas de dicionários para alunos e disciplinas, dicionários com chave composta utilizando a tupla (id_aluno, id_disciplina) em que o valor é a nota. As listas de dicionário simples utilizam a chave “ID” e o valor “nome” de forma intuitiva para navegar facilmente estes dados. Já os dicionários de chave composta utilizam a combinação dos ID’s para que não sejam necessários ciclos encadeados para encontrar a nota necessária, agrupando de forma atómica as notas dos alunos às disciplinas.

O comportamento do sistema é assegurado por um conjunto de funções bem definidas distribuídas pelos módulos:

Módulo logica.py

- `iniciar_dados()`: Sincroniza o estado inicial do programa populando as listas e o dicionário com os dados armazenados nos ficheiros CSV. Adicionalmente, injeta dados predefinidos se o sistema detetar uma primeira execução.
- `adicionar_aluno(nome)` / `adicionar_disciplina(nome)`: Processam e validam a introdução de novos registos, calculando o novo ID de forma incremental baseado no valor máximo existente.
- `remover_aluno(id_aluno)` / `remover_disciplina(id_disc)`: Removem a entidade especificada dos dicionários em memória e invocam a limpeza das notas associadas.
- `atribuir_nota(id_aluno, id_disc, valor)`: Cria ou atualiza a associação da nota no dicionário de notas.
- `resumo_geral()`: Agrega os dados e quantifica-os, gerando um *ranking* ordenado de forma decrescente com base na média de cada estudante.

Módulo ficheiros.py

- `guardar_alunos(alunos)` / `guardar_disciplinas(disciplinas)` / `guardar_notas(notas)`: Abrem os respetivos ficheiros .csv em modo de escrita, estruturando os cabeçalhos e persistindo o estado atual em disco com codificação UTF-8.
- `carregar_alunos()` / `carregar_disciplinas()` / `carregar_notas()`: Efetuam a leitura dos ficheiros através do `csv.DictReader`, convertendo os tipos de dados textuais nativos do formato CSV.

Módulo interface.py

- `iniciar_aplicacao()`: Instância a janela principal, configura as suas propriedades físicas, como dimensões fixas e título, e expõe o menu principal.
- `menu_gestao_alunos()` / `menu_gestao_disciplinas()` / `menu_gestao_notas()` / `menu_relatorios()`: Geram janelas secundárias para a execução das tarefas específicas a cada módulo do sistema.

A robustez do software é, por sua vez, garantida através de múltiplos mecanismos de validação implementados nas camadas de lógica e interface, prevenindo falhas críticas:

O método `.strip()` é aplicado nas entradas de texto para remover espaços brancos desnecessários, impedindo o registo de nomes vazios ou nulos.

Antes de aceitar um novo aluno ou disciplina, o sistema faz uma verificação a maiúsculas e minúsculas, através do `.lower()`, garantindo que não existem registos redundantes (ex: impedir que "Matemática" e "matemática" coexistam).

Na interface, a conversão de strings para inteiro ID's ou decimais Notas é controlada por blocos `try-except ValueError`. Caso o utilizador introduza caracteres alfabéticos em campos numéricos, o programa não colapsa, mas antes exibe um aviso ao utilizador.

O registo de notas está estritamente limitado ao intervalo numérico $[0, 20]$. Qualquer valor que não se encontre compreendido neste intervalo origina uma mensagem de erro.

A interação com o utilizador baseia-se numa interface gráfica baseada em eventos, que se traduz numa experiência fluida e intuitiva para fácil navegação de qualquer utilizador.

Ao iniciar o programa, é apresentado o menu principal onde cada botão desencadeia a abertura de um outro ecrã dedicado. As janelas secundárias utilizam o método `grab_set()`, obrigando o utilizador a interagir e fechar o submenu ativo antes de poder regressar ao menu inicial.

A exibição e seleção de dados é centralizada através do componente `ttk.Treeview`, organizado em tabelas com cabeçalhos claros. De forma a otimizar a usabilidade da aplicação, foi implementada pequena janela com eventos virtuais `TreeviewSelect`. Quando um utilizador clica em cima de qualquer registo na lista de alunos ou disciplinas, o sistema captura automaticamente o ID desse elemento e preenche-o de imediato no campo de texto de correspondente. Reduzindo assim drasticamente o esforço do utilizador, que deixa de ter a necessidade de memorizar ou escrever manualmente os números de identificação.

Por fim, todo o feedback das operações é comunicado através de janelas `pop-up` nativas, que variam entre alertas informativos de SUCESSO, mensagens críticas de ERRO detalhadas e diálogos de confirmação Sim/Não para operações potencialmente destrutivas, tais como a remoção de dados ou o reiniciar completo do sistema.

PERSISTÊNCIA DE DADOS

Para garantir que os dados não se perdem entre utilizações do programa, foi implementado um sistema de armazenamento local que guarda o estado das listas e dicionários diretamente no disco rígido.

Para garantir que os dados manipulados não se perdem ao encerrar a aplicação, foi implementado um sistema de persistência local baseado em ficheiros de texto, CSV, através dos ficheiros `alunos.csv`, `disciplinas.csv` e `notas.csv`. Este formato foi escolhido pela sua leveza, facilidade de leitura e exportação para EXCEL, mas também pela integração nativa em Python através da biblioteca `csv`.

Sempre que ocorre uma alteração nos dados (adição, remoção ou atribuição de notas), as funções do módulo `ficheiros.py` abrem os respetivos ficheiros em modo de escrita, "w". A gravação utiliza a codificação UTF-8 para suportar corretamente os caracteres da língua portuguesa e o parâmetro `newline=""` para evitar linhas em branco. A primeira linha é reservada para o cabeçalho estrutural e as restantes para os registos em memória.

No arranque do programa, a função `iniciar_dados()` invoca a leitura dos ficheiros através do `csv.DictReader`. O sistema utiliza o método `os.path.exists()` para validar previamente a existência dos ficheiros: se não existirem na primeira execução, o programa, como forma de prevenir erros, injeta dados de exemplo; se existirem, carrega-os e efetua a conversão das strings do CSV em inteiros para os IDs e floats para as notas.

PESQUISA ORDENÇÃO E ESTATÍSTICA

Para os critérios de pesquisa por ID, a aplicação utiliza uma pesquisa sequencial em que o sistema percorre a lista de ID's comparando os até encontrar na lista o ID correspondente e retornando os dados pedidos ou `None` caso não exista. Para a pesquisa por nome, ela é efetuada insensivelmente face a maiúsculas e minúsculas, verificando se objeto de pesquisa está contigo em qualquer parte do nome do aluno ou disciplina.

Para a ordenação alfabética é utilizada a função nativa do Python, `sorted()`, convertendo os nomes para minúsculas com `.lower()` para garantir que nada interfira na ordem correta dos caracteres. Para a ordenação de ID's, o sistema simplesmente corre a lista e organiza os ID's de forma crescente. Por fim, em termos de ordenação, a aplicação recorre a `reverse=True` para que as médias mais altas sejam colocados no topo e as mais baixas subsequentemente.

Em termos de estatística, a média começa por fazer uma filtragem no dicionário de notas para extrair os valores numéricos pretendidos, logo de seguida utiliza `sum(valores) / len(valores)` calcular o valor da média. Caso um dado não tenha nota atribuída o programa devolve `None` para evitar erros de divisão por zero. Para o resumo dos totais a aplicação uso o método `len()` diretamente nas listas e dicionário para contabilizar o número de referências existentes.

TESTES REALIZADOS

Para garantir a fiabilidade, a integridade dos dados e o correto funcionamento da lógica do sistema, foi desenvolvido um módulo de testes unitários automatizados com recurso ao `pytest` e `assert`.

No início de cada teste, a função `setup_teste()` limpa o estado global em memória e elimina os ficheiros físicos. Adicionalmente, as constantes do módulo `ficheiros.py` são temporariamente redirecionadas para ficheiros de teste (ex: `teste_alunos.csv`), garantindo que a base de dados real do programa nunca é corrompida durante a execução dos testes. Cada teste utiliza `assert` para validar se o resultado devolvido por uma função coincide com o comportamento esperado.

O sistema valida se a introdução de um nome válido resulta em sucesso, enquanto nomes vazios ou tentativas de duplicar registos existentes disparam mensagens de erro controladas.

Através de `assert`, confirma-se que o ID é sempre incremental. O teste valida que, mesmo removendo o aluno com ID 1, o próximo aluno criado recebe o ID 3, garantindo que os identificadores nunca são reaproveitados, por uma questão de integridade e interoperabilidade dos dados.

Valida-se também alteração de nomes com sucesso se alterar para um nome que ainda não exista e erro para um nome já ocupado. E ainda eficácia da pesquisa sequencial por partes de texto contida nas listas e dicionários.

Seguindo uma lógica análoga à dos alunos, os testes com `assert` comprovam o bloqueio de nomes de disciplinas vazios ou repetidos, bem como o correto funcionamento do incremento automático de ID's

Valida-se se o sistema aceita notas válidas e rejeita através de mensagens de erro qualquer nota fora do intervalo $[0, 20]$, ou atribuições a alunos e disciplinas com IDs inexistentes.

O módulo de testes verifica também a componente crítica que é a integridade referencial. O programa atribui uma nota a um aluno e usa o assert para confirmar que a nota foi guardada em memória. Logo a seguir, o aluno é removido e um novo assert valida se a nota passou a ser None, comprovando que o sistema limpa automaticamente os dados órfãos.

DECLARAÇÃO DE USO DE IAGEN

Para auxiliar a realização do trabalho, utilizamos o Gemini e o Claude, sendo que nos ajudou ao desenvolvimento da interface gráfica e a transformação do pseudocódigo para o python, respetivamente. Após o uso das mesmas, foi verificado todo o código e posteriormente, corrigido/adaptado para uma melhor compreensão.

CONCLUSÃO

Em suma, podemos concluir que o Sistema de Gestão de Notas, concebido em Python e suportado pela biblioteca gráfica Tkinter, foi estruturado seguindo uma arquitetura modular rigorosa, o que permite segmentar todo os blocos de código em vários ficheiros *.py*.

Em termos funcionais o programa desenvolvido permite a atribuição e consulta de classificações, onde também é possível calcular as médias dos alunos, de cada disciplina e um ranking ordenado da melhor para a pior nota.

No futuro, poderá existir uma migração para um sistema de gestão de base de dados relacional, o que teria uma maior escalabilidade e segurança. Poderá também haver uma transição para web, na qual se irá substituir a interface Tkinter para uma framework em Django, onde este estaria disponível remotamente a partir de qualquer dispositivo através de um navegador de internet.